MINI PROJECT REPORT

Performance Analysis of Machine Learning Models
Using L1 and L2 Regularization for Regression and Classification Tasks

Name	Lakshay Manchanda
SAP ID	590011784
Batch	B21
Program	B. Tech. CSE
Course	Applied Machine Learning
Course Code	CSAI2017P
Semester	IV
Name of the Faculty	Dr. Sahinur Laskar

1. Problem Definition

1.1 Objective

This project investigates the performance impact of L1 (Lasso) and L2 (Ridge) regularization on machine learning models applied to both regression and classification tasks.

In addition to analyzing model performance, this project critically evaluates the role of L1 and L2 regularization in controlling model complexity under correlated financial features. While transaction type classification is used as a multi-class benchmark problem, it is acknowledged that fraud detection (binary classification) is a more realistic application of the dataset. The chosen tasks allow controlled experimentation to isolate the effect of regularization on both regression and classification settings.

1.2 Tasks Defined

- Regression Task: Predict the log-transformed transaction amount (continuous target) using engineered financial features from the PaySim dataset.
- Classification Task: Predict the type of a financial transaction across 5 classes (CASH_IN, CASH_OUT, DEBIT, PAYMENT, TRANSFER).

1.3 Significance

Financial fraud detection is a high-stakes domain where both model interpretability and generalization are critical. Regularization ensures models do not overfit to training noise, which is especially important in high-dimensional, imbalanced financial data.

2. Data Understanding

2.1 Dataset Overview

The PaySim dataset is a synthetic financial transaction dataset generated using real patterns from a mobile money service in Africa. It simulates one month of financial activity and is a standard benchmark for fraud detection ML research, sourced from Kaggle.

Dataset Link - https://www.kaggle.com/datasets/ealaxi/paysim1

Property	Details
Source	Kaggle — PaySim Mobile Money Simulator
Total Records	6,362,620 transactions
Original Columns	11 (step, type, amount, nameOrig, oldbalanceOrg, newbalanceOrig, nameDest, oldbalanceDest, newbalanceDest, isFraud, isFlaggedFraud)
Sample Used	50,000 rows — 10,000 per transaction type (stratified sampling)
Transaction Types	CASH_IN, CASH_OUT, DEBIT, PAYMENT, TRANSFER (5 classes)
Fraud Ratio	~0.13% of all transactions are fraudulent (severely imbalanced)
Missing Values	None — 0 null values across all columns

2.2 EDA Key Observations

- Transaction amount is heavily right-skewed. Log1p transformation applied to normalize for regression.
- Fraud occurs exclusively in TRANSFER and CASH_OUT types — strong class-feature relationship.
- CASH_OUT and PAYMENT dominate by volume; DEBIT is the rarest transaction type.
- Balance differential features show strong separation between legitimate and fraudulent transactions.
- No missing values or duplicates — no imputation required.

3. Data Preprocessing

3.1 Sampling

Due to the full dataset size (6.3M rows), a stratified sample of 50,000 records was drawn at 10,000 transactions per type, preserving class distributions while ensuring computational feasibility.

3.2 Feature Engineering

Six new features were created to enrich the predictive signal:

Feature	Formula	Rationale
balanceOrigDiff	newbalanceOrig − oldbalanceOrg	Net change in sender balance
balanceDestDiff	newbalanceDest − oldbalanceDest	Net change in receiver balance
origBalanceZero	1 if oldbalanceOrg == 0	Zero-balance sender (fraud signal)
destBalanceZero	1 if oldbalanceDest == 0	Zero-balance receiver indicator
amountLog	log1p(amount)	Normalised amount (regression target)
surplusOrig	oldbalanceOrg − amount	Remaining sender balance post-transaction

3.3 Encoding & Scaling

- LabelEncoder applied to 'type' column, mapping 5 transaction types to integer labels 0–4. Name columns dropped.
- StandardScaler applied independently to regression and classification feature matrices. Critical for regularized models — unscaled features cause disproportionate penalization.

3.4 Final Feature Sets

- Regression: 12 features — all numeric columns excluding raw 'amount' (replaced by amountLog as target).
- Classification: 11 features — all numeric columns excluding 'amount' and 'amountLog'.

4. Train–Test Split

An 80/20 stratified split was applied. For classification, stratification preserves transaction type proportions in both sets. random_state=42 ensures reproducibility across all experiments.

Task	Total Samples	Training (80%)	Test (20%)
Regression — Predict log(Amount)	50,000	40,000	10,000
Classification — Predict Transaction Type	50,000	40,000	10,000

- Regression: RidgeCV and LassoCV used alpha selection via 5-fold CV from 100 log-spaced values in $[10^{-3}, 10^{3}]$.
- Classification: 5-Fold Stratified KFold used to produce robust macro F1 CV scores for all 13 models.

5. Model Training

5.1 Core Concept — L1 vs L2

- L1 (Lasso): Penalizes sum of |coefficients|. Creates corner solutions at axes — exact-zero coefficients likely - automatic feature selection.
- L2 (Ridge): Penalizes sum of coefficient². Smooth shrinkage toward zero but never exactly zero — all features retained.
- ElasticNet: Combines both penalties. l1_ratio=0.5 balances feature selection with coefficient shrinkage.

5.2 Regression Models

Model	Regularization	Configuration
Linear Regression (OLS)	None	Baseline — no penalty
Ridge — Optimal alpha	L2	alpha via RidgeCV (5-fold)
Ridge — alpha = 0.1	L2	Under-regularized comparison
Ridge — alpha = 10.0	L2	Over-regularized comparison
Lasso — Optimal alpha	L1	alpha via LassoCV (5-fold)
Lasso — alpha = 0.01	L1	Light penalty, minimal sparsity
Lasso — alpha = 0.5	L1	Heavy penalty, aggressive sparsity
ElasticNet	L1 + L2	l1_ratio=0.5, optimal alpha from LassoCV

5.3 Classification Models

In Logistic Regression, C = 1/λ. Smaller C = stronger regularization.

Model	Regularization	Configuration
Logistic Regression (No Reg)	None	penalty=None, solver=lbfgs, multinomial
Logistic Regression L2 (C=1.0)	L2	Default strength, solver=lbfgs
Logistic Regression L2 (C=0.01)	L2	Strong L2 regularization
Logistic Regression L1 (C=1.0)	L1	Sparse model, solver=saga
Logistic Regression L1 (C=0.1)	L1	Strong L1, aggressive sparsity
SGD Classifier (L2)	L2	Modified Huber loss, alpha=0.001, n_jobs=-1
Decision Tree (No Reg)	None	max_depth=None (fully grown)
Decision Tree (Regularized)	Structural	max_depth=8, min_samples_split=20, min_samples_leaf=10
Random Forest (Ensemble)	Ensemble	100 estimators, max_depth=10, n_jobs=-1

Gaussian Naïve Bayes	None	Probabilistic model, var_smoothing=1e-9
KNN (k=5)	None	n_neighbors=5, n_jobs=-1; trained on 5k stratified subsample
LinearSVC (SVM-linear)	L2	max_iter=2000, CalibratedClassifierCV wrapper; trained on 5k stratified subsample
Bagging Classifier (20 DTs)	Ensemble	n_estimators=20, base=DecisionTreeClassifier, n_jobs=-1
XGBoost (Gradient Boosting)	Ensemble	n_estimators=100, max_depth=6, learning_rate=0.1, eval_metric=mlogloss

6. Prediction & Evaluation

6.1 Regression Results

All models predicted log(transaction amount) on the held-out test set. Metrics: R^2, RMSE, MAE, and Overfit Gap (Train R^2 − Test R^2).

Model	Test R^2	Test RMSE	Test MAE	Overfit Gap
Linear Regression (OLS)	0.2377	1.8713	1.5008	~0.0232 (low baseline)
Ridge — Optimal alpha	0.2366	1.8727	1.5048	~0.0222 (reduced)
Ridge — alpha = 0.1	0.2377	1.8713	1.5008	~0.0232
Ridge — alpha = 10.0	0.2377	1.8713	1.5008	~0.0232
Lasso — Optimal alpha	0.2367	1.8725	1.5047	~0.0224 (reduced)
Lasso — alpha = 0.01	0.2368	1.8724	1.5044	~0.0225
Lasso — alpha = 0.5	0.0931	2.0411	1.7494	~0.0021 (underfitting)
ElasticNet (L1+L2)	0.2370	1.8721	1.5036	~0.0226

6.2 Classification Results

All metrics computed using macro-averaging across 5 classes (treats all classes equally). CV F1 from 5-Fold Stratified KFold.

Model	Accuracy	Precision	Recall	F1 Macro	CV F1
Logistic Reg (No Reg)	~0.8746	~0.8762	~0.8784	~0.8751	~0.8706
Logistic Reg L2 (C=1.0)	~0.8371	~0.8428	~0.8372	~0.8376	~0.8336
Logistic Reg L2 (C=0.01)	~0.7691	~0.7679	~0.7710	~0.7639	~0.7614
Logistic Reg L1 (C=1.0)	~0.8425	~0.8483	~0.8428	~0.8432	~0.8380
Logistic Reg L1 (C=0.1)	~0.8367	~0.8422	~0.8368	~0.8372	~0.8327
SGD Classifier (L2)	~0.7790	~0.7645	~0.7815	~0.7654	~0.7649
Decision Tree (No Reg)	~0.8402	~0.8430	~0.8438	~0.8434	~0.8435
Decision Tree (Regularized)	~0.8781	~0.8868	~0.8817	~0.8786	~0.8751
Random Forest (Ensemble)	~0.8798	~0.8919	~0.8830	~0.8800	~0.8765
Gaussian Naïve Bayes	~0.6353	~0.7010	~0.6477	~0.6225	~0.6232

KNN (k=5)	~0.7965	~0.7969	~0.7989	~0.7951	~0.7948
LinearSVC (SVM-linear)	~0.8067	~0.7992	~0.8082	~0.8021	~0.7865
Bagging Classifier (20 DTs)	~0.8729	~0.8766	~0.8767	~0.8747	~0.8695
XGBoost (Gradient Boosting)	~0.8874	~0.8946	~0.8910	~0.8881	~0.8827

6.3 Confusion Matrix Analysis (5×5)

- A 5×5 confusion matrix was plotted for all 13 classification models.
- Random Forest and Decision Tree produced near-perfect diagonal matrices — almost all transaction types correctly identified.
- Logistic Regression models showed moderate confusion between similar types (CASH_IN vs PAYMENT), reducing macro P/R/F1.
- Strongly regularized models (L1 C=0.1, L2 C=0.01) showed more off-diagonal errors — confirming that over-regularization degrades classification.

6.4 Coefficient Analysis (L1 vs L2 Effect)

- Unregularized Logistic Regression: Largest coefficients — some features with very high magnitude.
- L1 (Lasso): Several coefficient groups driven to near-zero — feature selection demonstrated clearly in visualization.
- L2 (Ridge): All coefficients retained but uniformly shrunk, producing a smoother, balanced magnitude profile.

7. Comparative Analysis

7.1 Regression

- At optimal alpha, Ridge (L2) and Lasso (L1) perform nearly identically on test R^2 and RMSE, both marginally outperforming OLS on the overfit gap.
- Critical distinction: Lasso (L1) drove multiple feature coefficients to exactly zero (feature selection). Ridge retained all features with small non-zero coefficients.
- Lasso with high alpha (0.5) degraded performance severely (Test $R^2 \approx 0.093$), demonstrating that over-regularization causes underfitting.
- Alpha sensitivity analysis confirmed the sweet spot: too low alpha → overfitting; too high alpha → underfitting.
- ElasticNet matched the optimal Lasso/Ridge performance — offering a flexible hybrid at no accuracy cost.

7.2 Classification

- XGBoost (Gradient Boosting) achieved the highest F1 macro (~0.8881), followed closely by Random Forest (~0.8800), both being non-linear ensemble models not directly comparable to regularized linear models.
- Among linear models, Logistic Regression L1 (C=1.0) and L2 (C=1.0) performed equally (~0.83 F1), showing both types provide similar benefit at moderate strength.
- Stronger regularization (C=0.01 for L2, C=0.1 for L1) significantly degraded performance, especially for minority classes.
- Regularized Decision Tree (≈ 0.88 F1) performed nearly as well, even better as the unconstrained tree (≈ 0.84 F1), demonstrating that structural regularization effectively controls overfitting.

7.3 L1 vs L2 Summary

Criterion	No Regularization	L1 (Lasso)	L2 (Ridge)	ElasticNet
Feature Selection	No	Yes (zeros)	No	Partial
Multicollinearity	Poor	Moderate	Excellent	Good
Overfitting Control	None	Strong	Strong	Strong
Interpretability	Moderate	High (sparse)	Moderate	Moderate
Best Use Case	No redundancy	Few relevant features	All features useful	Uncertain relevance

8. Conclusion

- This study demonstrates that L1 and L2 regularization play a critical role in controlling model complexity and improving generalization, particularly in high-dimensional financial datasets with correlated features. While both techniques reduce overfitting, their functional behavior differs significantly and must be selected based on the problem context.

- In regression tasks, Ridge (L2) and Lasso (L1) achieved nearly identical predictive performance at optimal regularization strengths, indicating that regularization does not necessarily improve accuracy but stabilizes model behavior by reducing variance. However, L1 regularization provides a distinct advantage through inherent feature selection, producing sparse and interpretable models—an important consideration in financial domains where explainability is essential.

- In classification tasks, moderate regularization improved model stability, but excessive regularization led to clear underfitting, particularly degrading minority class performance. This highlights that regularization strength is not universally beneficial and must be carefully tuned using cross-validation.

- Importantly, non-linear ensemble models such as XGBoost and Random Forest significantly outperformed all linear models, suggesting that while regularization improves linear models, it cannot compensate for their inability to capture complex feature interactions. Therefore, regularization should be viewed as a **complementary technique**, not a substitute for model selection.

- Overall, the results indicate:
    1. Use **L1 (Lasso)** when feature selection and interpretability are priorities.
    2. Use **L2 (Ridge)** when all features are relevant and multicollinearity is present.
    3. Avoid excessive regularization, as it leads to underfitting.
    4. Prefer **ensemble methods** for real-world financial prediction tasks where non-linearity dominates.

- Finally, a key limitation of this study is the use of a balanced sampled dataset, which does not fully reflect the extreme class imbalance of real-world fraud detection. Future work should extend this analysis to imbalanced learning techniques and binary fraud classification to better align with practical deployment scenarios.